

Relazione di progetto

Labirinto dei Settori

Applicazione client-server in C/POSIX

Componenti del gruppo: Mattia Lemma - N86005170

Architettura: Client-server su socket TCP

Build: Makefile, Docker, Docker Compose

19 maggio 2026

Indice

1	Introduzione	2
2	Guida d'uso	2
2.1	Requisiti	2
2.2	Compilazione nativa	2
2.3	Avvio del server	3
2.4	Avvio del client	3
2.5	Comandi disponibili nel client	4
2.6	Esecuzione con Docker	4
3	Protocollo applicativo	5
3.1	Principi generali	5
3.2	Comandi dal client al server	5
3.3	Risposte e notifiche dal server	5
3.4	Codifica di mappe, posizioni e punteggi	6
3.5	Esempio di sessione applicativa	7
4	Dettagli implementativi	7
4.1	Organizzazione modulare	7
4.2	Framing robusto su stream TCP	8
4.3	Invio completo dei buffer	8
4.4	Server single-process con select	9
4.5	Sessioni client e disconnessione	9
4.6	Database utenti in memoria	10
4.7	Stato di gioco e proprietà delle celle	10
4.8	Spawn e movimento	11
4.9	Nebbia di guerra e viste della mappa	11
4.10	Aggiornamenti periodici e fine partita	12
4.11	Client interattivo	12
4.12	Limiti noti	12
5	Conclusioni	12

1 Introduzione

Il progetto Labirinto dei Settori realizza un gioco multiplayer a griglia con architettura client-server. Ogni giocatore si collega a un server centrale, registra un profilo, effettua il login e si muove all'interno di una mappa per conquistare celle. La logica della partita è interamente mantenuta dal server: il client non decide autonomamente se una mossa sia valida, non calcola i punteggi finali e non conserva una copia autoritativa della mappa. Questo aspetto è importante perché evita divergenze tra giocatori diversi e rende il server l'unico punto in cui vengono applicate le regole di gioco.

La partita si svolge su una griglia di dimensione fissa pari a 30×15 . Le celle possono essere libere, occupate da muri oppure conquistate da un giocatore. Il server sceglie casualmente una tra quindici mappe statiche compilate nel sorgente e assegna a ogni giocatore uno spawn casuale su una cella libera e non occupata. Durante il gioco, una mossa valida sposta il giocatore e assegna la proprietà della nuova cella al suo slot. Al termine del tempo configurato, il server calcola il vincitore contando il numero di celle possedute da ciascun giocatore.

L'applicazione è volutamente essenziale dal punto di vista delle dipendenze: usa solo la libreria standard C e le API POSIX necessarie per socket, `select(2)`, terminale e gestione dei file descriptor. La compilazione nativa è gestita da un `Makefile`; in alternativa sono presenti un `Dockerfile` e un file `compose.yaml` che permettono di avviare server e client in container.

2 Guida d'uso

2.1 Requisiti

Per l'esecuzione nativa sono necessari un ambiente Linux o compatibile POSIX, `gcc` e `make`. Il progetto usa socket, `select(2)`, `termios` e altre interfacce POSIX; per questo motivo l'ambiente di riferimento naturale è Linux. La build Docker usa come base `ubuntu:24.04` e installa `build-essential` e `make`.

La struttura principale del progetto è la seguente:

- `src/server`: contiene il server, il database utenti in memoria e la logica di gioco.
- `src/client`: contiene il client, la gestione dell'input e l'interfaccia testuale.
- `src/common`: contiene codice condiviso per rete, protocollo e parsing numerico.
- `Makefile`: produce gli eseguibili `bin/server` e `bin/client`.
- `Dockerfile` e `compose.yaml`: permettono l'esecuzione containerizzata.

2.2 Compilazione nativa

Dalla radice del progetto è sufficiente eseguire:

```
make
```

Il `Makefile` compila i moduli comuni, quelli del server e quelli del client con le opzioni:

```
-std=c11 -D_POSIX_C_SOURCE=200112L -Wall -Wextra -Wpedantic -O2 -g -I  
src
```

Gli oggetti intermedi vengono creati sotto `build/`; gli eseguibili finali vengono creati sotto `bin/`:

```
bin/server  
bin/client
```

Per eliminare build ed eseguibili:

```
make clean
```

Sono presenti anche due target di comodita:

```
make run-server  
make run-client
```

Il primo avvia il server sulla porta 4242, con durata di partita pari a 300 secondi e aggiornamento globale ogni 5 secondi. Il secondo avvia un client verso 127.0.0.1:4242.

2.3 Avvio del server

Il server richiede almeno la porta TCP su cui ascoltare. Accetta inoltre due argomenti opzionali: durata della partita e periodo degli aggiornamenti globali.

```
./bin/server <porta> [durata_secondi] [periodo_secondi]
```

Esempio:

```
./bin/server 4242 300 5
```

Gli argomenti hanno il seguente significato:

- **porta**: porta numerica di ascolto, validata nel range 1-65535.
- **durata_secondi**: durata complessiva della partita; se omessa vale 300. Il **main** del server la accetta nel range 10-86400.
- **periodo_secondi**: intervallo tra due broadcast globali; se omesso vale 5. Il valore ammesso è nel range 1-3600.

Il server non legge da **stdin**: una volta avviato rimane in ascolto, accetta connessioni, gestisce i comandi dei client e termina automaticamente allo scadere della durata configurata. Al termine invia a tutti i client collegati un messaggio di fine partita con vincitore e punteggi.

2.4 Avvio del client

Il client richiede host e porta del server:

```
./bin/client <host> <porta>
```

Esempio su macchina locale:

```
./bin/client 127.0.0.1 4242
```

Se il server gira su una macchina remota, 127.0.0.1 non deve essere usato: occorre indicare l'indirizzo IP o il nome DNS raggiungibile del server, per esempio:

```
./bin/client 203.0.113.10 4242
```

Il client apre una connessione TCP verso il server usando **getaddrinfo()** e prova gli endpoint restituiti finché una connessione riesce. Una volta collegato, mostra un'interfaccia testuale con stato della connessione, eventi recenti, vista locale, mappa globale se disponibile e riga di comando.

2.5 Comandi disponibili nel client

Dopo la connessione, il client accetta i seguenti comandi:

Comando	Effetto
<code>register <nickname></code> <code><password></code>	Registra un nuovo profilo nel database utenti in memoria del server.
<code>login <nickname></code> <code><password></code>	Autentica l'utente e lo inserisce nella partita.
<code>move up</code>	Richiede uno spostamento verso l'alto.
<code>move down</code>	Richiede uno spostamento verso il basso.
<code>move left</code>	Richiede uno spostamento a sinistra.
<code>move right</code>	Richiede uno spostamento a destra.
<code>users, list, l</code>	Richiede la lista degli utenti attivi con simbolo e coordinate.
<code>local</code>	Richiede la vista locale 11×11 centrata sul giocatore.
<code>global</code>	Richiede la mappa globale dei territori e le posizioni attive.
<code>help, h</code>	Mostra/richiama la guida integrata nell'interfaccia.
<code>quit, q</code>	Invia la richiesta di uscita e chiude il client.

In modalita interattiva i tasti `w`, `a`, `s`, `d` vengono letti immediatamente, senza dover premere Invio, e corrispondono rispettivamente a su, sinistra, giu e destra. Questa scelta è implementata disattivando temporaneamente la modalità canonica del terminale tramite `termios`. Alla chiusura del client il terminale viene ripristinato.

2.6 Esecuzione con Docker

Per costruire l'immagine:

```
docker build -t territory-conquest .
```

Per avviare il server in un network dedicato:

```
docker network create tc-net
docker run --rm --name tc-server --network tc-net \
-p 4242:4242 territory-conquest ./bin/server 4242 300 5
```

Per avviare un client nello stesso network:

```
docker run --rm -it --network tc-net \
territory-conquest ./bin/client tc-server 4242
```

Con Docker Compose, il server può essere avviato con:

```
docker compose up --build server
```

Un client può essere aperto con:

```
docker compose run --rm client
```

Per aprire più giocatori è sufficiente eseguire il comando del client più volte in terminali diversi.

3 Protocollo applicativo

3.1 Principi generali

Il protocollo usato tra client e server è un protocollo applicativo testuale. Non coincide con TCP/IP: TCP fornisce soltanto il canale affidabile e ordinato di byte, mentre il progetto definisce sopra tale canale un insieme di messaggi, comandi, risposte, formati e codici di errore.

Ogni messaggio applicativo è una riga di testo terminata da `\n`. Il framing è quindi basato sul carattere newline. Sia client sia server accumulano i byte ricevuti in un buffer e considerano completo un messaggio solo quando trovano `\n`. Questa scelta è necessaria perché TCP è uno stream: una `send()` può essere ricevuta in più `recv()`, oppure più righe possono arrivare insieme.

I messaggi client-server iniziano con il prefisso `C2S_`; i messaggi server-client iniziano con il prefisso `S2C_`. I campi sono separati da spazi o tabulazioni e vengono tokenizzati con la funzione comune `proto_split`. Il limite massimo di riga è `PROTO_MAX_LINE`, pari a 16384 byte; il numero massimo di token considerati è `PROTO_MAX_TOKENS`, pari a 32.

3.2 Comandi dal client al server

Messaggio	Significato applicativo
<code>C2S_REGISTER <nick> <pass></code>	Registra un utente. Nickname e password devono essere non vuoti, lunghi al massimo 31 caratteri e composti solo da lettere, numeri, underscore o trattino.
<code>C2S_LOGIN <nick> <pass></code>	Autentica l'utente e lo associa a uno slot giocatore. Se il login riesce, il server assegna simbolo, posizione iniziale e invia la vista locale.
<code>C2S_MOVE UP</code>	Richiede una mossa verso l'alto.
<code>C2S_MOVE DOWN</code>	Richiede una mossa verso il basso.
<code>C2S_MOVE LEFT</code>	Richiede una mossa verso sinistra.
<code>C2S_MOVE RIGHT</code>	Richiede una mossa verso destra.
<code>C2S_LIST_USERS</code>	Richiede l'elenco dei giocatori attivi.
<code>C2S_LOCAL_MAP</code>	Richiede la vista locale del giocatore autenticato.
<code>C2S_GLOBAL_MAP</code>	Richiede la mappa globale dei territori e le posizioni attive.
<code>C2S_QUIT</code>	Richiede la disconnessione ordinata.

Il client traduce i comandi utente in questi messaggi. Per esempio, `register mario segreta` diventa `C2S_REGISTER mario segreta`; il tasto `w` o il comando `move up` diventano entrambi `C2S_MOVE UP`.

3.3 Risposte e notifiche dal server

Messaggio	Significato applicativo
<code>S2C_OK CONNECTED</code>	Connessione accettata. Viene inviato subito dopo l'accept.
<code>S2C_OK REGISTERED</code>	Registrazione completata con successo.

Messaggio	Significato applicativo
S2C_OK LOGGED_IN <nick> <symbol> <x> <y>	Login completato. Il server comunica nickname, simbolo stabile e coordinate iniziali.
S2C_OK MOVED <x> <y>	Mossa valida; il server comunica la nuova posizione.
S2C_OK BYE	Uscita accettata.
S2C_LOCAL_MAP <w> <h> <map>	Vista locale centrata sul giocatore. Nel progetto $w=11$ e $h=11$.
S2C_GLOBAL_UPDATE <w> <h> <map> <positions>	Aggiornamento globale, inviato su richiesta o periodicamente. Nel progetto $w=30$ e $h=15$.
S2C_USERS <positions>	Lista compatta dei giocatori attivi.
S2C_GAME_OVER <winner> <score> <scores>	Fine partita, con vincitore, punteggio vincente e riepilogo di tutti i punteggi.
S2C_ERR <code>	Errore applicativo. Il codice specifica il motivo.

I principali codici di errore prodotti dal server sono:

Errore	Condizione
BAD_SYNTAX	Messaggio con numero di token errato o forma non valida.
UNKNOWN_COMMAND	Messaggio non riconosciuto e senza prefisso C2S_.
NOT_AUTHENTICATED	Comando di gioco richiesto prima del login.
USER_EXISTS	Registrazione di un nickname già presente.
INVALID_CREDENTIALS	Nickname o password non rispettano i vincoli sintattici.
AUTH_FAILED	Login con credenziali errate.
ALREADY_AUTHENTICATED	Login richiesto da una sessione già autenticata.
USER_ALREADY_ONLINE	Lo stesso nickname è già attivo in partita.
GAME_FULL	Impossibile inserire il giocatore, ad esempio per assenza di celle libere.
OUT_OF_BOUNDS	Movimento fuori dai confini della mappa.
WALL	Movimento verso una cella muro.
OCCUPIED	Movimento verso una cella occupata da un altro giocatore attivo.
LINE_TOO_LONG	Riga ricevuta oltre il limite del buffer.
ENCODING_FAILED	Errore nella serializzazione di mappa, posizioni o punteggi.
SERVER_FULL	Server impossibilitato ad accettare nuovi client.

3.4 Codifica di mappe, posizioni e punteggi

Le mappe vengono inviate come una singola stringa testuale. Le celle della stessa riga sono separate da virgole, mentre le righe sono separate da slash. Una mappa 3×2 , per esempio, avrebbe forma:

```
. , P0 , . / P1 , # , .
```

I simboli applicativi usati nella codifica sono:

- `.`: cella libera o non posseduta.
- `@`: posizione del giocatore nella vista locale.
- `#`: muro scoperto dal giocatore nella vista locale.
- `~`: cella fuori mappa nella vista locale.
- `P0, P1, ...`: cella conquistata dal giocatore associato allo slot indicato.

La vista locale è privata: mostra una finestra 11×11 centrata sul giocatore, include i muri che quel giocatore ha scoperto e usa `~` per le celle fuori dai bordi. La mappa globale è pubblica ma intenzionalmente meno informativa: mostra soltanto la proprietà delle celle e non rivela la disposizione dei muri non scoperti.

Le posizioni sono codificate come una lista separata da virgole. Ogni elemento ha forma:

```
nickname:symbol:x:y
```

Esempio:

```
alice:P0:10:4,bob:P1:20:7
```

Se non ci sono giocatori attivi, la lista vale `-`. I punteggi finali seguono una codifica analoga, `nickname:punteggio`, separata da virgole; anche in questo caso `-` indica lista vuota.

3.5 Esempio di sessione applicativa

Una possibile sequenza di scambio è la seguente:

```
S2C_OK CONNECTED
C2S_REGISTER alice pass1
S2C_OK REGISTERED
C2S_LOGIN alice pass1
S2C_OK LOGGED_IN alice P0 12 8
S2C_LOCAL_MAP 11 11 ...
S2C_GLOBAL_UPDATE 30 15 ... alice:P0:12:8
C2S_MOVE UP
S2C_OK MOVED 12 7
S2C_LOCAL_MAP 11 11 ...
```

L'ordine mostra due aspetti importanti. Primo, il server può inviare più messaggi in risposta a un singolo comando: dopo un login riuscito invia conferma, vista locale e aggiornamento globale. Secondo, gli aggiornamenti globali non sono soltanto risposte dirette: il server li invia anche periodicamente a tutti i client autenticati.

4 Dettagli implementativi

4.1 Organizzazione modulare

Il codice è diviso in tre aree. La cartella `src/common` contiene funzionalità condivise: creazione e connessione dei socket, invio affidabile delle righe, tokenizzazione dei messaggi e parsing numerico. La cartella `src/server` contiene lo stato autoritativo: sessioni client, database utenti, mappa, giocatori, proprietà delle celle e timer di partita. La cartella `src/client` contiene la traduzione dei comandi utente nel protocollo, la ricezione delle risposte e l'interfaccia testuale.

Questa separazione permette di mantenere il protocollo in un modulo comune, evitando duplicazioni tra client e server. Per esempio, entrambe le parti usano `proto_make_line()` per garantire che ogni messaggio finisca con newline e `proto_split()` per dividere una riga in token.

4.2 Framing robusto su stream TCP

Una delle scelte più importanti riguarda la gestione dei messaggi. Poiché TCP non conserva i confini applicativi, non è corretto assumere che una chiamata a `recv()` corrisponda a un comando completo. Il server mantiene quindi un buffer per ogni sessione client e processa solo righe complete.

```
1 while (1)
2 {
3     char *line_break = memchr(session->inbuf, '\n', session->inbuf_len);
4     size_t line_length;
5     char complete_line[PROTO_MAX_LINE];
6
7     if (line_break == NULL)
8     {
9         break;
10    }
11    line_length = (size_t)(line_break - session->inbuf);
12    memcpy(complete_line, session->inbuf, line_length);
13    complete_line[line_length] = '\0';
14    memmove(session->inbuf, line_break + 1,
15            session->inbuf_len - line_length - 1);
16    session->inbuf_len -= line_length + 1;
17    dispatch_client_command(server, session_index, complete_line);
18 }
```

Listing 1: Estrazione di righe complete dal buffer del server, tratto da `server.c`.

Il client applica la stessa idea per le risposte del server. Questa simmetria rende il protocollo più affidabile: un aggiornamento globale molto lungo o più risposte ravvicinate non confondono il parser, purché ogni messaggio rispetti il limite massimo di riga.

4.3 Invio completo dei buffer

Anche l'invio richiede attenzione: `send()` può scrivere meno byte di quelli richiesti. Per questo il modulo comune `net.c` implementa un ciclo `net_send_all()` che continua finché tutto il buffer è stato inviato oppure si verifica un errore reale.

```
1 static int net_send_all(int fd, const char *buffer, size_t buffer_length
2 )
3 {
4     size_t bytes_sent = 0;
5
6     while (bytes_sent < buffer_length)
7     {
8         ssize_t sent_now = send(fd, buffer + bytes_sent,
9                                buffer_length - bytes_sent, 0);
10
11         if (sent_now < 0)
12         {
13             if (errno == EINTR)
14             {
15                 continue;
16             }
17             return -1;
18         }
19         if (sent_now == 0)
20         {
21             return -1;
22         }
23         bytes_sent += sent_now;
24     }
```

```

21     bytes_sent += (size_t)sent_now;
22 }
23 return 0;
24 }

```

Listing 2: Invio completo dei byte, tratto da `net.c`.

Questa funzione viene usata da `net_send_line()`, che spedisce righe già formate. Il server e il client ignorano inoltre `SIGPIPE`, così un tentativo di scrittura su socket chiuso non termina brutalmente il processo.

4.4 Server single-process con select

Il server è single-process e single-thread. Tutte le connessioni vengono multiplexate con `select(2)`: il set dei descrittori contiene sempre il socket di ascolto e tutti i socket client attivi. A ogni iterazione il server calcola anche un timeout, usato per svegliarsi al momento del prossimo aggiornamento globale o della fine partita.

```

1  FD_ZERO(&readable_fds);
2  FD_SET(server.listen_fd, &readable_fds);
3  for (session_index = 0; session_index < server.client_count; ++
      session_index)
4  {
5      if (server.clients[session_index].fd >= 0)
6      {
7          FD_SET(server.clients[session_index].fd, &readable_fds);
8          if (server.clients[session_index].fd > max_fd)
9          {
10             max_fd = server.clients[session_index].fd;
11         }
12     }
13 }
14
15 timeout.tv_sec = seconds_until_next_server_event(&server);
16 timeout.tv_usec = 0;
17 select_result = select(max_fd + 1, &readable_fds, NULL, NULL, &timeout);

```

Listing 3: Schema del ciclo principale del server, tratto da `server.c`.

Questa soluzione è coerente con l'obiettivo del progetto: evitare thread e sincronizzazione esplicita, mantenendo comunque la capacità di gestire più client contemporaneamente. Il limite principale è quello tipico di `select`: i file descriptor devono essere inferiori a `FD_SETSIZE`, e infatti il codice rifiuta connessioni con descriptor troppo alto.

4.5 Sessioni client e disconnessione

Ogni connessione è rappresentata da una `client_session_t`, che memorizza file descriptor, stato di autenticazione, id del giocatore e buffer di input. L'array delle sessioni cresce dinamicamente con `realloc`; quando un client si disconnette, lo slot può essere riutilizzato.

La disconnessione non cancella la storia del giocatore. Se un utente aveva conquistato celle, quelle celle restano assegnate al suo slot anche quando il client va offline. La funzione di disconnessione chiude il socket, segna il giocatore come inattivo e pulisce la sessione, ma non azzerla la proprietà delle celle.

```

1  static void disconnect_session(server_t *server, int session_index)
2  {
3      client_session_t *session = &server->clients[session_index];
4

```

```

5     if (session->fd >= 0)
6     {
7         close(session->fd);
8     }
9     if (session->authenticated && session->player_id >= 0)
10    {
11        game_remove_player(&server->game, session->player_id);
12    }
13    clear_client_session(session);
14 }

```

Listing 4: Disconnessione di una sessione, tratto da `server.c`.

Questa scelta consente di calcolare punteggi finali che tengono conto anche di giocatori non più connessi, ma che hanno effettivamente conquistato territorio durante la partita.

4.6 Database utenti in memoria

Il database utenti è un array dinamico di strutture `user_t`, ciascuna con nickname e password. Non esiste persistenza su file o database esterno: alla chiusura del server tutte le registrazioni vengono perse. La registrazione controlla tre condizioni: validità sintattica, assenza di duplicati e spazio disponibile nell'array.

La validazione delle credenziali accetta solo caratteri alfanumerici, underscore e trattino. La lunghezza massima di nickname e password è 31 caratteri. L'autenticazione è un confronto diretto tra stringhe memorizzate in memoria. In un sistema reale servirebbero hashing e persistenza, ma per questo progetto la scelta è coerente con l'obiettivo di concentrarsi su comunicazione, multiplexing e logica di gioco.

4.7 Stato di gioco e proprietà delle celle

Lo stato della partita è descritto dalla struttura `game_t`: contiene la matrice dei muri, la matrice dei proprietari e l'array dei giocatori. Ogni proprietario è indicato con l'indice dello slot giocatore; -1 rappresenta una cella senza proprietario.

```

1  typedef struct {
2      char nickname[NICK_MAX + 1];
3      int active;
4      int used;
5      char symbol[PLAYER_SYMBOL_MAX];
6      int x;
7      int y;
8      unsigned char discovered_walls[MAP_H][MAP_W];
9  } player_t;
10
11 typedef struct {
12     int wall[MAP_H][MAP_W];
13     int owner[MAP_H][MAP_W];
14     player_t *players;
15     size_t player_count;
16     size_t player_capacity;
17 } game_t;

```

Listing 5: Strutture principali del gioco, tratte da `game.h`.

La distinzione tra `active` e `used` è significativa. `active` indica che il giocatore è online; `used` indica che quello slot è stato assegnato almeno una volta. In questo modo uno slot può restare proprietario di celle anche se il giocatore non è connesso. Il simbolo testuale P0, P1, ... è stabile per slot e viene usato nelle mappe e nella UI.

4.8 Spawn e movimento

Quando un giocatore effettua il login, il server cerca una cella libera e non occupata. La funzione di spawn esegue due passaggi: prima conta tutte le celle valide, poi sceglie casualmente la k -esima cella libera. Questo evita di allocare una lista temporanea di coordinate.

Il movimento controlla in ordine: validità del giocatore, nuova coordinata, confini della mappa, presenza di muro e presenza di altri giocatori attivi. Solo se tutti i controlli passano il giocatore viene spostato e la cella diventa di sua proprietà.

```

1  if (!is_inside_map(nx, ny)) {
2      return -2;
3  }
4  if (game->wall[ny][nx]) {
5      p->discovered_walls[ny][nx] = 1;
6      reveal_walls_around_player(game, player_id);
7      return -3;
8  }
9  for (i = 0; i < game->player_count; ++i) {
10     if ((int)i != player_id && game->players[i].active &&
11         game->players[i].x == nx && game->players[i].y == ny) {
12         return -4;
13     }
14 }
15 p->x = nx;
16 p->y = ny;
17 game->owner[ny][nx] = player_id;
18 reveal_walls_around_player(game, player_id);
19 return 0;

```

Listing 6: Regole principali del movimento, tratte da `game.c`.

I codici di ritorno interni vengono tradotti dal server in messaggi applicativi S2C_OK MOVED, S2C_ERR OUT_OF_BOUNDS, S2C_ERR WALL o S2C_ERR OCCUPIED.

4.9 Nebbia di guerra e viste della mappa

Un dettaglio interessante è la gestione dei muri scoperti. La matrice dei muri è nota al server, ma non viene rivelata globalmente. Ogni giocatore possiede una matrice `discovered_walls`; quando si trova vicino a un muro o tenta di entrarci, quel muro viene marcato come scoperto solo per lui.

```

1  for (dy = -1; dy <= 1; ++dy) {
2      for (dx = -1; dx <= 1; ++dx) {
3          x = p->x + dx;
4          y = p->y + dy;
5          if (is_inside_map(x, y) && game->wall[y][x]) {
6              p->discovered_walls[y][x] = 1;
7          }
8      }
9  }

```

Listing 7: Rivelazione locale dei muri, tratta da `game.c`.

La vista locale è una finestra 11×11 centrata sul giocatore. Nella sua costruzione il server inserisce @ per la posizione corrente, # per i muri già scoperti, ~ fuori dalla mappa, il simbolo del proprietario se la cella è conquistata, altrimenti .. La mappa globale, invece, non mostra i muri: contiene solo proprietari e celle libere. Questo crea una distinzione netta tra informazione privata e informazione pubblica.

4.10 Aggiornamenti periodici e fine partita

Il server mantiene `start_time` e `next_update`. La funzione `seconds_until_next_server_event()` calcola il timeout di `select` confrontando la prossima scadenza di broadcast con la scadenza finale della partita. Se il tempo corrente supera `next_update`, il server invia a tutti i client autenticati un `S2C_GLOBAL_UPDATE`; se supera la durata totale, invia `S2C_GAME_OVER` e termina.

Il punteggio è calcolato contando quante celle hanno come proprietario lo slot di ciascun giocatore. Il vincitore è il giocatore `used` con punteggio massimo. In caso di assenza di giocatori, il server usa il simbolo - e punteggio 0.

4.11 Client interattivo

Il client usa a sua volta `select(2)` per attendere contemporaneamente due sorgenti: il socket del server e `stdin`. Questo permette di ricevere aggiornamenti periodici mentre l'utente sta digitando. In modalità terminale interattiva, il client passa a una modalità non canonica: i tasti rapidi `w/a/s/d` vengono inviati subito come movimento.

L'interfaccia viene ridisegnata interamente a ogni aggiornamento significativo. Lo stato visualizzato è contenuto in una struttura `ui_state_t`: connessione, nickname, simbolo, posizione, mappe serializzate, posizioni e registro degli ultimi eventi. Per gli eventi viene usato un ring buffer fisso di sei messaggi, scelta semplice che evita allocazioni dinamiche nella UI.

4.12 Limiti noti

Il progetto è funzionale rispetto agli obiettivi richiesti, ma presenta alcuni limiti consapevoli:

- Il database utenti è solo in memoria e non sopravvive al riavvio del server.
- Le password sono salvate in chiaro, scelta accettabile solo in un contesto didattico.
- Il server usa `select`, quindi il numero massimo di file descriptor è vincolato da `FD_SETSIZE`.
- Non esiste cifratura applicativa: la sicurezza del canale non è un obiettivo di questa implementazione.
- Le mappe sono statiche e compilate nel sorgente; non vengono caricate da file esterni.

Questi limiti non compromettono la dimostrazione dei concetti centrali del progetto: protocollo applicativo, gestione di più client, framing testuale, stato autoritativo lato server, aggiornamenti periodici e UI client reattiva.

5 Conclusioni

Il progetto realizza una vera applicazione client-server con protocollo applicativo esplicito e stato condiviso gestito centralmente. La parte più rilevante non è l'uso di TCP in se, ma la definizione del livello applicativo costruito sopra lo stream: comandi `C2S`, risposte `S2C`, framing a newline, codifica di mappe e posizioni, errori applicativi e notifiche periodiche.

Dal punto di vista implementativo, le scelte più significative sono il server single-process basato su `select`, la gestione robusta dei buffer di ricezione, l'invio completo dei messaggi, la separazione tra vista locale e vista globale, la persistenza degli slot giocatore durante le disconnessioni e l'interfaccia client capace di gestire input e rete contemporaneamente. Nel complesso il codice rimane compatto, portabile in ambiente POSIX e coerente con una consegna centrata su comunicazione client-server e coordinamento dello stato di gioco.